

Brent Foxworth Programming Assignment: Task List (Linked List)

Program Design:

This program implements an interactive **task list manager** using a **singly linked list**. It allows a user to add tasks at specific positions, delete tasks by position, display the total number of tasks, and print tasks in both forward and reverse order. The program enforces a maximum of **8 tasks** and continues running until the user chooses to quit.

Program Structure:

The program is structured into the following components:

1. **User Interface**
 - Displays a menu and prompts the user for input.
 - Controls program flow using a loop in main().
2. **Input Parsing and Validation**
 - Ensures inputs are non-empty, correctly formatted, and within valid ranges.
 - Prevents invalid positions and overflow beyond the maximum list size.
3. **Linked List Operations**
 - Handles insertion and deletion of tasks at specific positions.
 - Uses pointer manipulation to maintain list integrity.
4. **Output and Printing**
 - Displays task count.
 - Prints tasks in forward or reverse order.

Data Types Used:

struct node

Represents a single task in the list.

- **string task** – stores the task name.
- **node* next** – pointer to the next node in the list.

Constants

- **MAX_ITEMS** = 8 – defines the maximum number of tasks and the largest valid position.

Main Program Flow:

main()

- Initializes the linked list with `node* head = NULL`.
- Repeatedly:
 - Displays the menu.
 - Calls `performOption(&head)` to process the user's choice.
- Terminates when option **6 (Quit)** is selected.

displayMenu()

Prints the list of available user options.

performOption(node** head)

- Calls `optionValid()` to get a validated menu choice.
- Executes the selected option using a switch statement.
- Returns the chosen option to control the loop in `main()`.

Core Functions:

Menu and Option Validation

- `optionValid(node* head)` - Ensures the user selects exactly one integer between 1 and 6 and prevents adding tasks when the list is full.

Counting and Printing

- `countNodes(node* head)` – returns the number of nodes in the list.
- `printTaskCount(node* head)` – prints the total number of tasks.
- `printTasksForward(node* head)` – prints tasks in ascending position order.
- `printTasksReverse(node* head)` – prints tasks in reverse order using recursion.
- `printTasksForwardOption()` / `printTasksReverseOption()` – handle empty-list messaging for menu options 4 and 5.

Linked List Operations

- `insertAtStart(node** head, string task)` – inserts a task at position 1.
- `insertAtEnd(node** head, string task)` – inserts a task at the end of the list.
- `insertAtPosition(node** head, int position, string task)` – inserts a task at a specified position.
- `addNewNode(node** head)` – gathers validated input and calls the appropriate insert function.
- `removeNode(node** head)` – removes a task at a validated position and deallocates memory.

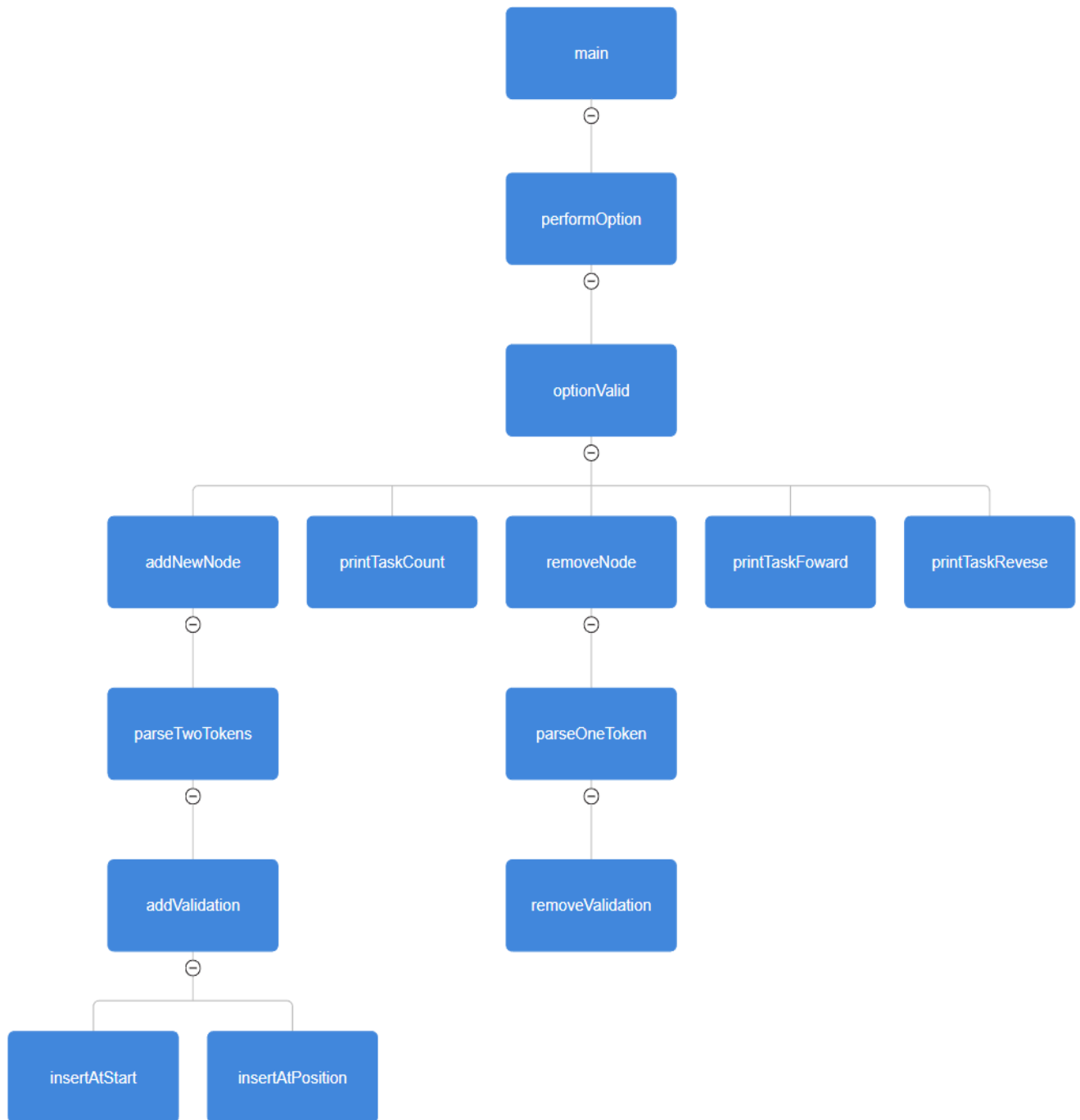
Input Parsing and Validation Utilities

- `stringSize(string)` – returns the length of a string without using `.size()`.
- `isInteger(string)` – checks whether a string contains only digits.
- `stringToInt(string)` – converts a numeric string to an integer.
- `parseOneToken()` – ensures exactly one input token is provided.
- `parseTwoTokens()` – ensures exactly two input tokens are provided.
- `addValidation()` – validates add positions and prevents gaps in the list.
- `removeValidation()` – validates delete positions.
- `hasNodeBefore()` / `hasNodeAt()` – verify valid list positions.

Summary:

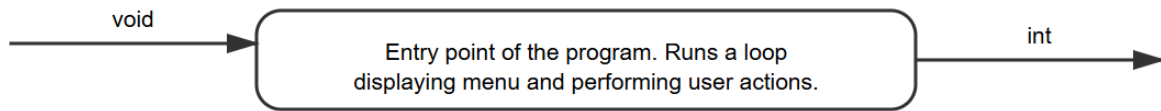
This program demonstrates structured use of **linked lists**, **manual input validation**, and **modular function design**. By separating validation, list manipulation, and output, the program ensures correctness, readability, and maintainability while providing a robust task-list interface for the user.

Hierarchical diagrams/ Structure charts

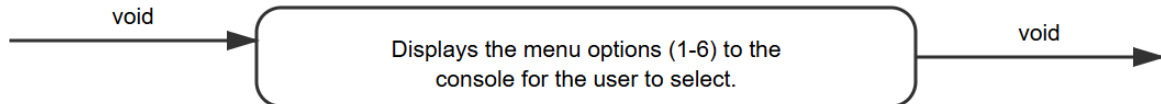


Procedure Specification

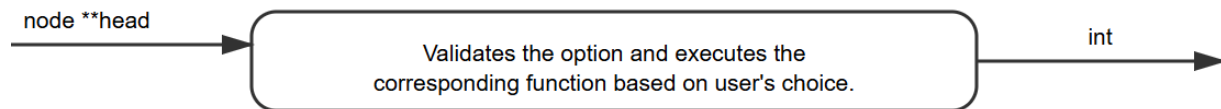
Function main



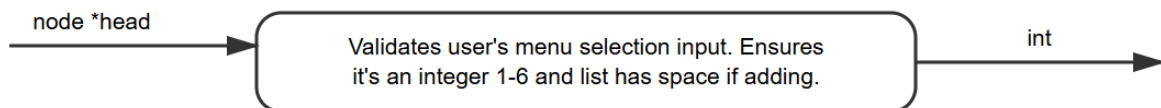
Function displayMenu



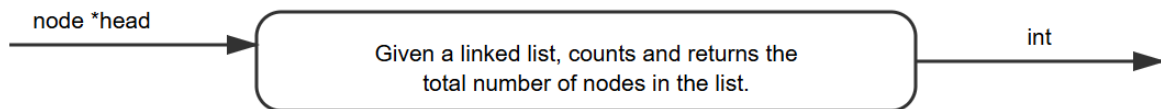
Function performOption



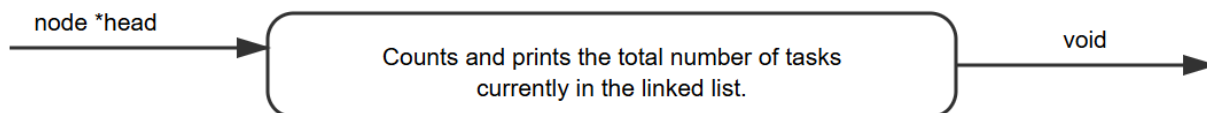
Function optionValid



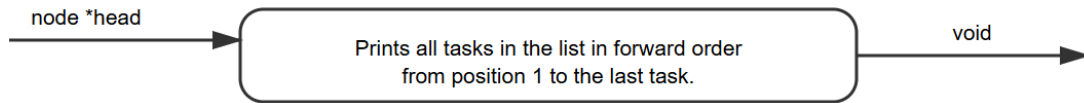
Function countNodes



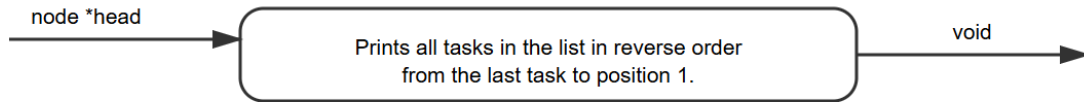
Function printTaskCount



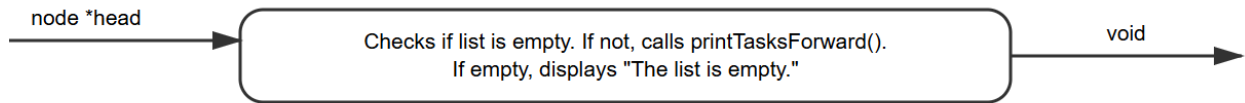
Function printTasksForward



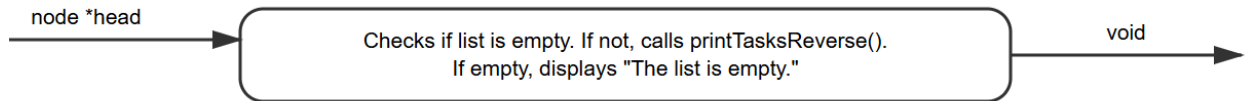
Function printTasksReverse



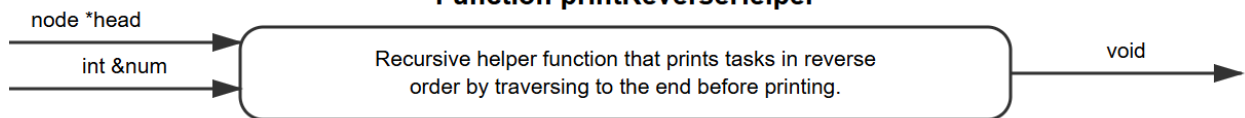
Function printTasksForwardOption



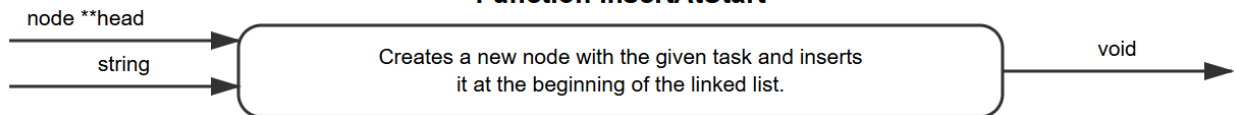
Function printTasksReverseOption



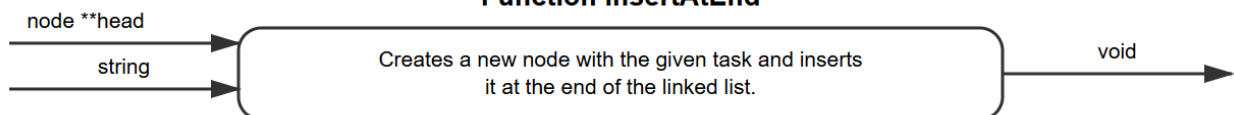
Function printReverseHelper

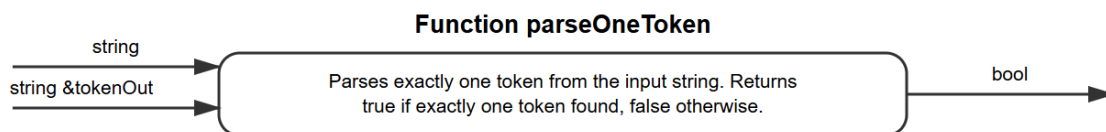
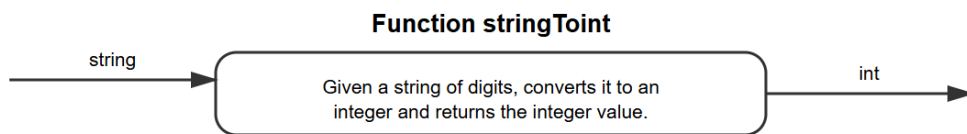
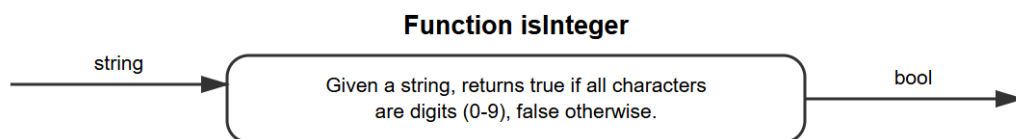
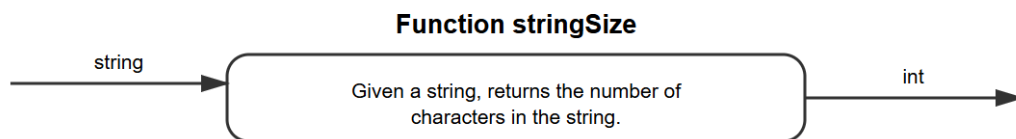
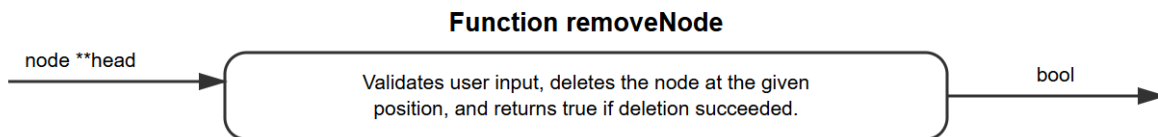
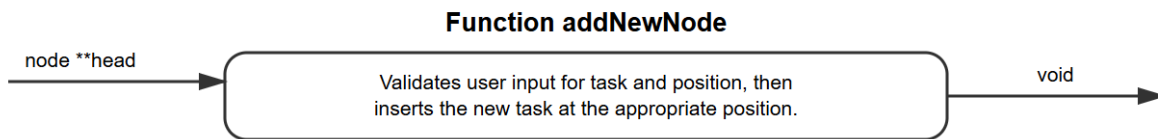
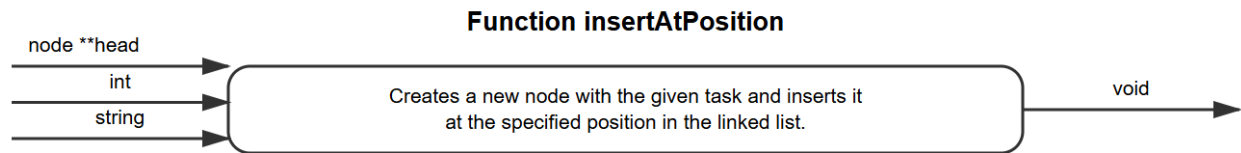


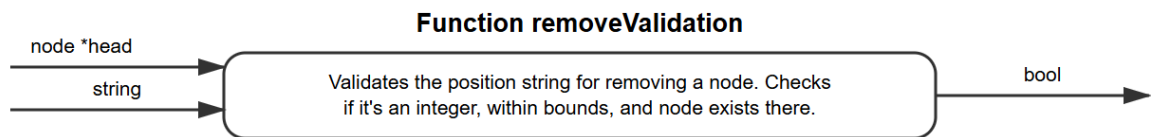
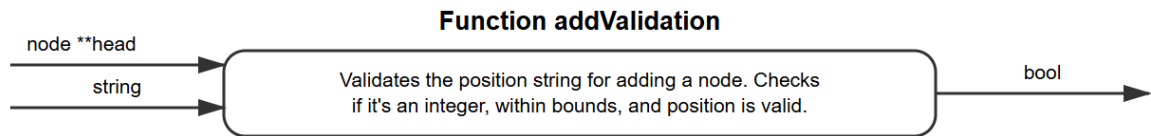
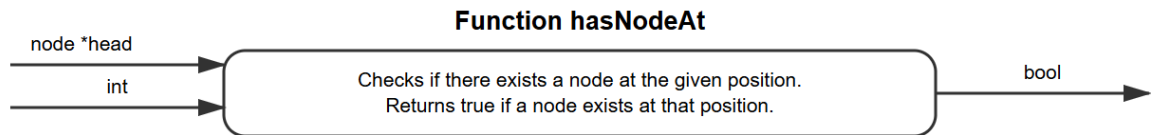
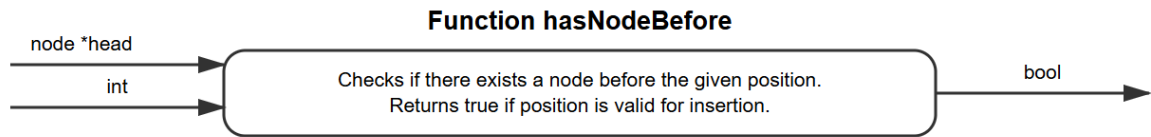
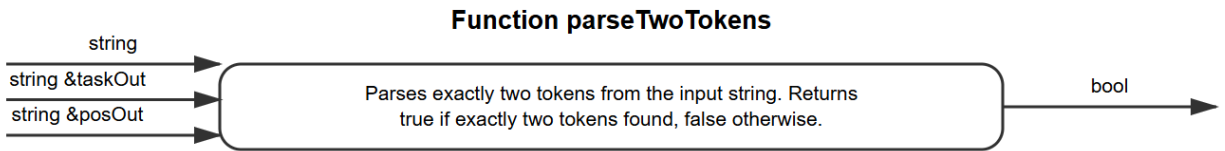
Function insertAtStart



Function insertAtEnd







Test Cases:

Main Menu Test Cases:

- 1 - Test if add option runs
- 2 - Test if delete option runs
- 3 - Test if program can accurately print the number of tasks in list
- 4 - Test if program can accurately print the tasks in the list
- 5 - Test if program can accurately print the tasks in the list in reverse
- 6 - Test if program ends

Note: 3-5 must be tested multiple times when list empty and where there a different task in the list

Add task Test Cases

- ab 8 - See if program allow this (should not)
- ab 1 - See if task can be added based on position
- cd 2 - See if task can be added based on position
- ef 3 - See if task can be added based on position
- gh 1 - See if task can be added based on position and push everything else back
- ab 1.5 - See if program crashes because 1.5 isn't a valid position
- ab ab - See if program crashes because ab isn't a valid position
- ab 0 - See if program crashes because 0 is not a valid position
- 9 ab - See if program crashes because the max task should be 8
- ab _ - See if program crashes because _ is not a valid position
- ab ?? - See if program crashes because ?? is not a valid position
- 1 2 3 4 - See if program crashes because this input isn't valid
- 1 ab - See if program crashes because this input isn't valid

Delete task Test Cases

Notes: for this example the current list is 1: ab 2: gh 3: ab 4: cd 5: ef

- 1 - See if ab is deleted and all other task are moved up 1
- 5 - See if program can delete at empty position (should not)
- 0 - See if program can delete from a position that does not exist (should not)
- 9 - See if program can delete from a position that does not exist (should not)
- A - See if program crashes because "A" is not a valid position
- _ - See if program crashes because "_" is not a valid position
- 1 2 3 - See if program crashes because this input isn't valid
- 1 ab - See if program crashes because this input isn't valid
- ab 1 - See if program crashes because this input isn't valid

Results of Test Cases

Main Menu Test Cases:

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
```

```
Select an option (1-6): 1
Please enter task and position: ab 1
Number of tasks: 1
1: ab
1: ab
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
```

```
Select an option (1-6): 2
Enter position to remove: 1
```

```
ab was deleted.
Number of tasks: 0
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
```

```
Select an option (1-6): 3
Number of tasks: 3
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
```

```
Select an option (1-6): 4
1: ab 2: cd 3: ef
```

```
1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
```

```
Select an option (1-6): 5
3: ef 2: cd 1: ab
```

Add task Test Cases:

1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit

Select an option (1-6): 1

Please enter task and position: ab 8

Invalid position.

Please enter task and position: ab 1

Number of tasks: 1

1: ab

1: ab

-
1. Add task X to the list at a particular position p
 2. Delete task X from the task list at position q
 3. Print the number of tasks in the task list
 4. Print all tasks in the task list in the order of position
 5. Print all tasks in the task list in reverse order of position
 6. Quit

Select an option (1-6): 1

Please enter task and position: cd 2

Number of tasks: 2

1: ab 2: cd

2: cd 1: ab

-
1. Add task X to the list at a particular position p
 2. Delete task X from the task list at position q
 3. Print the number of tasks in the task list
 4. Print all tasks in the task list in the order of position
 5. Print all tasks in the task list in reverse order of position
 6. Quit

Select an option (1-6): 1

Please enter task and position: ef 3

Number of tasks: 3

1: ab 2: cd 3: ef

3: ef 2: cd 1: ab

-
1. Add task X to the list at a particular position p
 2. Delete task X from the task list at position q
 3. Print the number of tasks in the task list
 4. Print all tasks in the task list in the order of position
 5. Print all tasks in the task list in reverse order of position
 6. Quit

Select an option (1-6): 1

Please enter task and position: gh 1

Number of tasks: 4

1: gh 2: ab 3: cd 4: ef

4: ef 3: cd 2: ab 1: gh

-
1. Add task X to the list at a particular position p
 2. Delete task X from the task list at position q
 3. Print the number of tasks in the task list
 4. Print all tasks in the task list in the order of position
 5. Print all tasks in the task list in reverse order of position
 6. Quit

Select an option (1-6): 1

Please enter task and position: ab 1.5

Invalid position.

Please enter task and position: ab ab

Invalid position.

Please enter task and position: ab 0

Invalid position.

Please enter task and position: 9 ab

Invalid position.

Please enter task and position: ab __

Invalid position.

Please enter task and position: ab ??

Invalid position.

Please enter task and position: 1 2 3 4

Invalid input: enter exactly TWO items.

Please enter task and position: 1 ab

Invalid position.

Please enter task and position: -|

Delete Task Test Cases:

- ```

1. Add task X to the list at a particular position p
2. Delete task X from the task list at position q
3. Print the number of tasks in the task list
4. Print all tasks in the task list in the order of position
5. Print all tasks in the task list in reverse order of position
6. Quit
```

```
Select an option (1-6): 2
Enter position to remove: 1
```

```
ab was deleted.
Number of tasks: 4
1: gh 2: ab 3: cd 4: ef
4: ef 3: cd 2: ab 1: gh
```

- ```
-----  
1. Add task X to the list at a particular position p  
2. Delete task X from the task list at position q  
3. Print the number of tasks in the task list  
4. Print all tasks in the task list in the order of position  
5. Print all tasks in the task list in reverse order of position  
6. Quit
```

```
Select an option (1-6): 2  
Enter position to remove: 5  
Invalid position.
```

```
Enter position to remove: 0  
Invalid position.
```

```
Enter position to remove: 9  
Invalid position.
```

```
Enter position to remove: a  
Invalid position.
```

```
Enter position to remove: _  
Invalid position.
```

```
Enter position to remove:  
Invalid input: you entered nothing.
```

```
Enter position to remove: 1 2 3  
Invalid input: enter exactly ONE item.
```

```
Enter position to remove: 1 ab  
Invalid input: enter exactly ONE item.
```

```
Enter position to remove: ab 1  
Invalid input: enter exactly ONE item.
```

```
Enter position to remove: |
```

